

جلسه 9

تابع در پایتون

تابع در پایتون گروهی از عبارت‌های مرتبط است که یک کار مشخص را انجام می‌دهند. توابع کمک می‌کنند تا برنامه به بخش‌های کوچک‌تر و دانه‌بندی شده‌ای (ماژولار | modular) شکسته شود. هرچه برنامه بزرگ و بزرگ‌تر شود، تابع‌ها به سازمان‌یافته‌تر و قابل مدیریت شدن آن کمک می‌کنند. علاوه بر این، توابع مانع از تکرار برنامه‌نویسی برای یک کار واحد می‌شوند و کد را قابل استفاده مجدد می‌کنند.

```
1 def function_name(parameters):  
2     """docstring"""  
3     statement(s)
```

آنچه در کد بالا نمایش داده شده، تعریف یک تابع است که شامل مولفه‌های زیر می‌شود:

1. کلیدواژه def علامت آغاز «سرآیند (Header)» تابع است.
2. نام تابع (function_name) که به صورت یکتا، تابع را مشخص کند. نام‌گذاری تابع در پایتون از قواعدی مشابه با قواعد نام‌گذاری (Identifiers) تبعیت می‌کند.
3. پارامترها (آرگومان‌ها) که به وسیله آن‌ها، مقادیر به تابع پاس داده می‌شوند. آرگومان‌ها اختیاری هستند.
4. یک علامت نقل قول یا دو نقطه (:) برای تعیین پایان هدر.
5. داک استرینگ (docstring) اختیاری برای تشریح آنکه تابع چه کاری انجام می‌دهد.
6. یک یا تعداد بیشتری دستور پایتون که بدنه تابع را تشکیل می‌دهند. دستورها باید دارای سطح دندان‌گذاری یکسانی باشند (معمولاً ۴ فاصله).
7. یک دستور return اختیاری برای بازگرداندن یک مقدار از تابع.

```
1 def greet(name):
2     """This function greets to
3     the person passed in as
4     parameter"""
5     print("Hello, " + name + ". Good morning!")
```

```
>>> print(greet.__doc__)
This function greets to
the person passed into the
name parameter
```

داکاسترینگ (Docstring)

اولین رشته پس از عنوان تابع را Docstring می‌گویند. Docstring مخففی برای Documentation String است. از docstring برای ارائه تعریفی کوتاه از عملکرد تابع استفاده می‌شود. اگرچه استفاده از docstring اختیاری است، اما مستندسازی یک کار مهم در برنامه‌نویسی محسوب می‌شود.

حو دستور return در پایتون

این دستور می‌تواند شامل دستوری باشد که ارزیابی و مقدار آن بازگردانده می‌شود. اگر هیچ عبارتی در دستور وجود نداشت یا خود دستور return درون یک تابع نبود، تابع، شی None را باز می‌گرداند. مثال زیر در این راستا، قابل توجه است.

```
1 def absolute_value(num):
2     """This function returns the absolute
3     value of the entered number"""
4
5     if num >= 0:
6         return num
7     else:
8         return -num
9
10 # Output: 2
11 print(absolute_value(2))
12
13 # Output: 4
14 print(absolute_value(-4))
```

دامنه و طول عمر متغیرها

دامنه متغیر (Scope of Variable)، بخشی از برنامه است که متغیر در آن شناخته شده است. پارامترها و متغیرهای تعریف شده درون یک تابع از بیرون از همان تابع، در دسترس نیستند.

بنابراین، دارای «دامنه محلی (Local Scope)» هستند. طول عمر یک متغیر، مدت زمانی است که متغیر در حافظه قرار دارد. طول عمر یک متغیر درون تابع، به اندازه مدت زمان اجرای آن تابع است. متغیرهای درون تابع پس از بازگشت (return) از تابع از بین می‌روند. در ادامه، مثالی برای نمایش دامنه متغیر درون یک تابع ارائه شده است.

```
def my_func():  
    x = 10  
    print("Value inside function:",x)  
  
x = 20  
my_func()  
print("Value outside function:",x)
```

در قطعه کد بالا، می‌توان مشاهده کرد که مقدار x به طور مقدماتی برابر با ۲۰ است. حتی اگر تابع my_func() مقدار x را به ۱۰ تغییر دهد، مقدار بیرون از تابع را تحت تاثیر قرار نمی‌دهد. این موضوع به این دلیل است که متغیر x درون تابع (برای تابع محلی است) از متغیر بیرون تابع متفاوت است. از سوی دیگر، متغیرهای بیرون تابع از درون آن تابع قابل مشاهده هستند. در واقع، متغیرهای بیرون از یک تابع، دارای دامنه سراسری (global scope) هستند. شایان توجه است که می‌توان این مقادیر را از درون تابع خواند، اما نمی‌توان آن‌ها را تغییر داد (نوشت). به منظور ویرایش مقدار متغیرهای بیرون از تابع، این متغیرها باید با استفاده از کلیدواژه global تعریف شوند.

*args, **kwargs در پایتون

کلمه کلیدی «*args» در پایتون نشان می‌دهد هیچ محدودیتی در تعداد آرگومان‌های ارسال شده به تابع وجود ندارد. البته این آرگومان‌ها نباید به شکل کلید-مقدار باشند. در حالی که «**kwargs» به معنای آن است که تابع می‌تواند آرگومان‌هایی با ساختار کلید-مقدار بگیرد. از این نوع آرگومان فقط در توابعی استفاده می‌شود که با ساختمان داده دیکشنری کار می‌کنند. در برنامه نویسی اگر بخواهیم از کدهای خود بارها و به صورت تکراری استفاده کنیم، باید آن‌ها را به شکل تابع بنویسیم. توابع همیشه وظیفه ثابتی را تکرار می‌کنند. در زمان تعریف تابع، می‌توانیم آرگومان‌های مختلفی به عنوان ورودی مشخص کنیم. اما اگر بخواهیم، تابعی تعداد نامحدودی آرگومان دریافت کند، باید از ساختارهای *args و **kwargs در پایتون استفاده کنیم.

*args در پایتون

فرض کنیم مسئله‌ای داده شده است. این مسئله از ما می‌خواهد ماشین حساب بسازیم. در مسئله، گفته شده که ماشین حساب مورد نظر، فقط عملیات ضرب انجام می‌دهد. در ادامه مطلب، تابع ضرب ماشین حساب را پیاده‌سازی کرده‌ایم. برای پیاده‌سازی عملیات ضرب، می‌توانیم از از تابع ساده‌ای استفاده کنیم. در کادر زیر، کدهای مربوط به تابع ضرب دو عدد را مشاهده می‌کنید.

```
1 def multiply(num1,num2):  
2     return num1*num2  
3 print("product:", multiply(2,3))
```

بعد از اجرای کد بالا خروجی به شکل `product:6` نمایش داده می‌شود.

الان باید ماشین حساب خود را طوری تغییر دهیم که ۳ عدد در ورودی دریافت کند. سپس حاصل ضرب آن‌ها را به بیرون برگرداند. بنابراین تابع جدیدی – مانند کد زیر – می‌نویسیم.

```
1 def multiplyThreeNumbers(num1, num2, num3):  
2     return num1*num2*num3  
3  
4 print("product:",multiplyThreeNumbers(1, 2, 3))
```

بعد از اجرای کد بالا خروجی به شکل زیر نمایش داده می‌شود.

```
product:6
```

همین الان هم می‌توانیم مشکل را تشخیص دهیم. اگر بخواهیم برنامه تعداد بیشتری عدد را در یکدیگر ضرب کند، در نهایت مجبور می‌شویم توابع زیادی بنویسیم. در چنین مواقعی با استفاده از `*args` مشکل را حل می‌کنیم. برای حل کردن این مسئله می‌توانیم با کمک آرگومان `*args` تکه‌کد ساده و انعطاف‌پذیری بنویسیم.

```
1 def multiplyNumbers(*numbers):
2     product=1
3     for n in numbers:
4         product*=n
5     return product
6
7 print("product:",multiplyNumbers(4,4,4,4,4,4))
```

بعد از اجرای کد بالا خروجی زیر نمایش داده می‌شود.

```
product:4096
```

در این بخش باید به دو نکته مهم اشاره کنیم.

- **نکته اول:** منظور از کلمه کلیدی آن است که آرگومان‌ها نمی‌توانند دیکشنری باشند. زیرا ساختار [دیکشنری‌های پایتون](#) به شکل «جفت کلید-مقدار» (Key-Value Pair) «است. آرگومان‌های `*args` فقط می‌توانند عدد یا رشته باشند.
- **نکته دوم:** کلمه «`args`» فقط نقش شناسه را بازی می‌کند. می‌توان آن را با هر چیز مربوطی جایگزین کرد. مسئله مهم، وجود داشتن نماد ستاره قبل از نام آرگومان است.

kwargs در پایتون

همین طور که اشاره شد از `*args` برای ارسال تعداد نامحدودی پارامتر به توابع، استفاده می‌کنیم. این پارامترها نباید کلمه کلیدی باشند. منظور از کلمه کلیدی ساختارهایی هستند که داده‌ها را به شکل جفت‌های کلید-مقدار نگهداری می‌کنند. برای نمونه می‌توانیم به دیکشنری‌های پایتون اشاره کنیم. «`**kwargs`» به برنامه نویسان کمک می‌کند تعداد نامحدودی آرگومان از جنس کلمه کلیدی به تابع ارسال کنند. در زبان برنامه نویسی پایتون، این کلمه‌های کلیدی به شکل دیکشنری مرتب شده و استفاده می‌شوند. پایتون، تمام کلماتی را که قبل از آن‌ها نماد دو ستاره «`**`» قرار گرفته است، به عنوان آرگومان کلمه کلیدی می‌شناسد.

```
1 def makeSentence(*words):  
2     sentence=''  
3     for word in words.values():  
4         sentence+=word  
5     return sentence  
6  
7 print('Sentence:', makeSentence(a='Kwargs ',b='are ', c='awesome!'))
```

بعد از اجرای کد بالا خروجی زیر نمایش داده می‌شود.

```
Sentence: Kwargs are awesome!
```

در تابع `makeSentence` در حال پیمایش بر روی دیکشنری هستیم. بنابراین برای استفاده از مقادیر ذخیره شده در این ساختار باید از تابع `values()` استفاده کنیم. در غیر این صورت فقط کلیدها به بیرون برگشت داده می‌شود و نه مقادیر.

در کادر زیر، مثال دیگری درباره روش استفاده از `**kwargs` را بررسی کرده‌ایم.

```
1 def whatTechTheyUse(**kwargs):
2     result = []
3     for key, value in kwargs.items():
4         result.append("{} uses {}".format(key, value))
5     return result
6
7 print(whatTechTheyUse(Google='Angular', Facebook='react', Microsoft='.NET'))
```

بعد از اجرای کد بالا خروجی زیر نمایش داده می‌شود.

```
['Google uses Angular', 'Facebook uses react', 'Microsoft uses .NET']
```

استفاده از `*args` و `**kwargs` در پایتون برای فراخوانی تابع

تا اینجای کار، متوجه معنی و کارکرد کلمات «`*args`» و «`**kwargs`» شده‌ایم. در این بخش از مطلب، تابعی طراحی کرده‌ایم که از هر دوی این کلمات استفاده کند. در زمان طراحی این تابع، ترتیب قرار گرفتن این آرگومان‌ها مهم است. در تعریف تابع، همیشه `*args` باید قبل از `**kwargs` قرار بگیرد.

در کادر زیر مثال ساده‌ای درباره تابعی نوشته‌ایم که از آرگومان‌های استاندارد، `args*` و `kwargs**` استفاده می‌کند.

```
1 def printingData(codeName, *args, **kwargs):  
2     print("I am ", codeName)  
3     for arg in args:  
4         print("I am arg: ", arg)  
5     for keyWord in kwargs.items():  
6         print("I am kwarg: ", keyWord)  
7  
8 printingData('007', 'agent', firstName='James', lastName='Bond')
```

بعد از اجرای کد بالا خروجی زیر نمایش داده می‌شود.

```
I am 007  
I am arg: agent  
I am kwarg: ('firstName', 'James')  
I am kwarg: ('lastName', 'Bond')
```


فضای نام در پایتون (Namespace)

فضای نام (Namespace) مجموعه‌ای از نام‌های نمادین تعریف شده در لحظه، به همراه اطلاعاتی درباره شیئی است که هر نام به آن ارجاع می‌دهد. می‌توانید فضای نام را همچون یک دیکشنری (یا نگاشت) در نظر بگیرید که در آن کلیدها همان نام‌های اشیاء و مقادیر، خود آن اشیاء هستند.

در یک برنامه پایتون، چهار نوع فضای نام وجود دارد:

- Built-In

- Global

- Enclosing

- Local

قانون LEGB برای جستجوی نام‌ها با بازگشت به پرسش مطرح شده در بخش پیشین، فرض کنید که از درون یک تابع داخلی به X ارجاع می‌دهید. در این حالت، پایتون به ترتیب زیر به دنبال X می‌گردد:

- محلی (Local): پایتون به دنبال X در داخل تابع درونی می‌گردد. اگر X را در آنجا پیدا نکند، به سراغ حوزه (scope) دربرگیرنده می‌رود.
- محدوده دربرگیرنده (Enclosing): در مرحله بعد، پایتون به دنبال X در محدوده تابع دربرگیرنده می‌گردد. اگر آن را در آنجا نیابد، به سراغ محدوده سراسری (global scope) می‌رود.
- Global: پایتون به دنبال X در محدوده سراسری (global scope) می‌گردد. اگر آن را پیدا نکند، به سراغ محدوده داخلی (built-in scope) می‌رود.
- داخلی (Built-in): در نهایت، پایتون به دنبال X در محدوده داخلی (built-in scope) می‌گردد. اگر X را در آنجا پیدا نکند، استثنای NameError را صادر می‌کند.

Generator در پایتون چیست؟

از Generator در پایتون برای ساخت مجموعه‌ای از داده‌ها استفاده می‌شود. این تابع به جای [عبارت return در پایتون](#)، داده‌های خود را با عبارت `yield` برمی‌گرداند. توابع Generator تمام عناصر خود را یکجا تولید نمی‌کنند. بلکه هر بار که اجرا می‌شوند فقط یک عنصر از توالی مورد نظر را تولید کرده و به خروجی ارسال می‌کنند.

قدرت ویژه-Generator ها به خاطر وجود کلمه کلیدی `yield` است. می‌دانیم که در توابع سنتی، دستور `return` مقداری را در خروجی برگردانده و سپس کار تابع را به طور کامل متوقف می‌کند. اما دستور `yield` هر بار که تابع اجرا می‌شود، فقط یک مقدار برمی‌گرداند. سپس اجرای تابع را به صورت موقت، متوقف می‌کند `yield`. تمام مشخصات تابع را در زمان توقف به خاطر می‌سپارد. وقتی Generator کار خود را دوباره شروع کند، دقیقاً از همان جایی کار را ادامه می‌دهد که قبلاً متوقف کرده بود.

مثال ساده ای از-Generator ها در پایتون

برای درک بهتر موضوع باید روش نوشتن کدهای Generator و استفاده از آن را بررسی کنیم. در کادر زیر، نمونه‌ای از توابع Generator را پیاده‌سازی کرده‌ایم. این تابع وظیفه دارد `n` عدد صحیح تولید بکند.

```
1 def generate_integers(n):
2     for i in range(n):
3         yield i # Pauses here and returns i
4
5 # Using the generator
6 for num in generate_integers(5):
7     print(num)
```

بعد از اجرای کدهای بالا خروجی پایین تولید شده و در کنسول پایتون نمایش داده می‌شود.

```
0
1
2
3
4
```

توابع Generator

توابع Generator مانند توابع معمولی پایتون تعریف می‌شوند. تنها تفاوت آن‌ها در استفاده از کلمه کلیدی `yield` به جای دستور `return` در پایتون است. وقتی از کلمه `yield` استفاده کنیم، در خروجی تابع، شیء `Generator` برگشت داده می‌شود. این شیء پیمایش‌پذیر - تکرارگر - است.

```
1 def count_up_to(n):
2     count = 1
3     while count <= n:
4         yield count
5         count += 1
6
7 # Using the generator
8 counter = count_up_to(5)
9 for num in counter:
10     print(num)
```

بعد از اجرای کدهای بالا خروجی پایین تولید شده و در کنسول پایتون نمایش داده می‌شود.

```
1
2
3
4
5
```

در مثال بالا، وقتی تابع `count_up_to()` اجرا کردیم، در خروجی شیء `Generator` برگردانده شد. هر بار که حلقه `for` به دنبال مقدار جدیدی می‌گردد، تابع اجرا می‌شود تا به دستور `yield` برسد. در آن لحظه، مقدار فعلی متغیر `count` به بیرون از تابع ارسال می‌شود. سپس وضعیت فعلی تابع و تمام متغیرهای آن ذخیره شده و منتظر شروع دور بعدی حلقه می‌شوند. وقتی حلقه دوباره اجرا شود، این تابع کار خود را دقیقاً از همان جایی ادامه می‌دهد که دفعه پیش متوقف شده بود.

عبارت‌های Generator

عبارت‌های Generator، به کدهای کوتاه و فشرده‌ای هستند که با آن‌ها تابع Generator تعریف می‌کنیم. این عبارت‌ها تقریباً ساختاری شبیه به «List Comprehension» دارند. اما در آن‌ها به جای «براکت» یا «[]» از پرانتز «()» استفاده می‌شود.

```
1 # List comprehension (eager evaluation)
2 squares_list = [x**2 for x in range(5)] # [0, 1, 4, 9, 16]
3
4 # Generator expression (lazy evaluation)
5 squares_gen = (x**2 for x in range(5))
6
7 # Using the generator
8 for square in squares_gen:
9     print(square)
```

بعد از اجرای کدهای بالا خروجی پایین، در کنسول پایتون نمایش داده می‌شود.

```
0
1
4
9
16
```